Componente front-end del proyecto formativo y proyectos de clase

Sistema de selección de personal

GA7-220501096-AA4-EV03

Laura Tatiana Ciro Botero

Servicio Nacional de Aprendizaje - SENA

Tecnología en Análisis y Desarrollo de Software

Ficha 3235898

Instructora Adriana Milena Rangel

2026

Introducción y objetivos

La empresa de servicios generales necesita organizar candidatos, consultar vacantes y conocer el avance de cada postulación. Esta evidencia desarrolla la interfaz diseñada en EV02 y la conecta con una API de Spring Boot creada para el proyecto. Las operaciones se guardan en una base relacional del servidor; no dependen de una lista simulada en el navegador.

El instrumento solicita concordancia con los prototipos, entrega de archivos y repositorio, navegación funcional y cumplimiento de los requisitos iniciales (Servicio Nacional de Aprendizaje [SENA], s. f.). Este documento explica la implementación e identifica dónde comprobar cada punto.

Objetivo general

Implementar y verificar un componente front-end en React y Vite para gestionar candidatos y postulaciones mediante una API REST de Spring Boot.

Objetivos específicos

• Convertir el diseño EV02 en componentes reutilizables con nombres PascalCase.

• Conectar registro, consulta, edición y seguimiento con el servidor.

• Aplicar validaciones, comentarios, formato consistente y control de versiones.

• Comprobar navegación, permisos y manejo de errores con pruebas automáticas.

Requisitos y relación con los prototipos

Tabla 1

Correspondencia del diseño con la implementación

Origen o requisito	Implementación verificable
GA5: encabezado, menú y pie	LayoutPrincipal conserva encabezado azul #004080, navegación visible y pie institucional.
GA5: acceso y formularios	AccesoUsuario y FormularioCandidato presentan etiquetas, campos, validación y mensajes.
EV02: consultar candidatos	PanelSeleccion, BuscadorCandidatos y ListaCandidatos permiten búsqueda y detalle.
EV02: registrar y corregir	FormularioCandidato se reutiliza en creación y edición.
EV02: postulación y estado	FormularioPostulacion y EstadoPostulacion vinculan persona y vacante; cada proceso tiene su estado.
EV02: historial y roles	HistorialEstado muestra responsables. El coordinador consulta; el analista modifica.
Adaptación a pantallas pequeñas	El menú se reorganiza y los formularios se adaptan; la tabla conserva desplazamiento horizontal.

Nota. Se conserva la estructura visual del prototipo GA5 y se concreta el dominio definido en EV02. El menú genérico de servicios se adapta a Vacantes. Ayuda explica el acceso; no se implementa envío de correos de recuperación. No se afirma una reproducción idéntica de todas las pantallas genéricas de GA5.

Alcance: aplicación web adaptable. No incluye aplicación móvil nativa, nómina, contratación ni administración de usuarios. Seleccionado no significa contratado.

Arquitectura y organización del código

React divide la interfaz en componentes. Los eventos del formulario y los botones llaman funciones; estas funciones consultan o actualizan el servidor y después modifican la pantalla (React, s. f.). Vite ejecuta el entorno de desarrollo y genera la versión compilada (Vite, s. f.).

Figura 1

Flujo de datos implementado



Nota. Elaboración propia. React se comunica por HTTP/JSON; el servidor consulta la base mediante JDBC y sentencias parametrizadas.

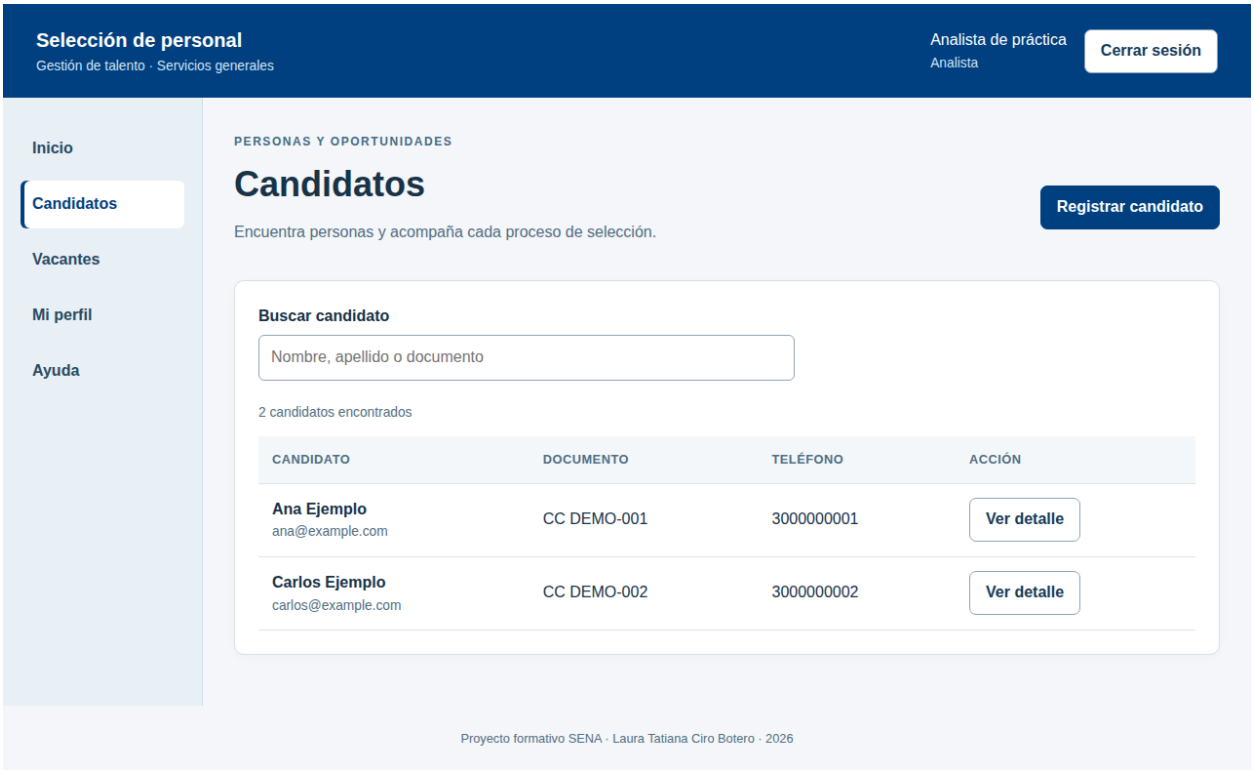
Carpeta	Responsabilidad
frontend/src/components	Formularios, listas, campos y mensajes reutilizables.
frontend/src/pages	Pantallas de candidatos, detalle, edición y vacantes.
frontend/src/context	Sesión y permisos de presentación.
frontend/src/services	Peticiones a la API y validaciones compartidas.
backend/src/main/java	Controlador REST, servicio, seguridad y validación.
backend/src/main/resources	Esquema SQL y perfiles H2/MySQL.
tests y src/test	Pruebas unitarias, de integración y de navegador.

Consulta y navegación de candidatos

Después de ingresar, el analista encuentra la lista de candidatos. Puede buscar por nombre, apellido o documento, abrir el detalle y registrar una nueva persona. El menú conserva acceso a Inicio, Candidatos, Vacantes, Mi perfil y Ayuda.

Figura 2

Listado ejecutado con conexión a Spring Boot



Nota. Captura real de Chromium durante las pruebas automatizadas en GitHub Actions. Los registros Ana Ejemplo y Carlos Ejemplo son datos ficticios del entorno académico.

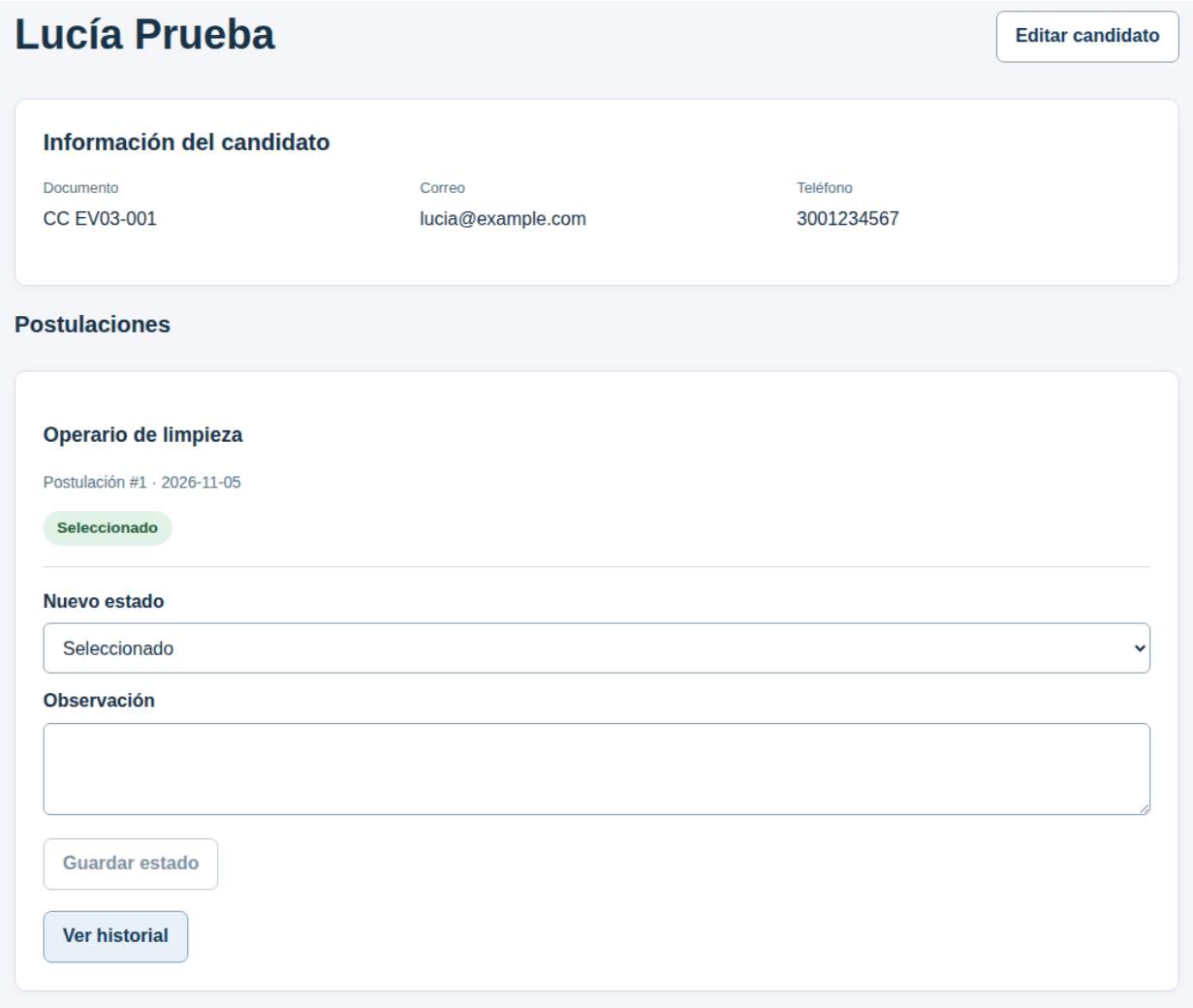
La búsqueda muestra el total de resultados, un aviso cuando no hay coincidencias y un mensaje con opción de reintentar si el servidor no responde. Los enlaces cambian la ruta; recargar conserva la pantalla y vuelve a consultar el servidor.

Postulación, estado e historial

El analista registra al candidato y elige una vacante abierta. El servidor impide repetir la misma postulación. Para cambiar el estado se exige una observación y se registra automáticamente el responsable de la sesión.

Figura 3

Postulación e historial después de guardar el cambio



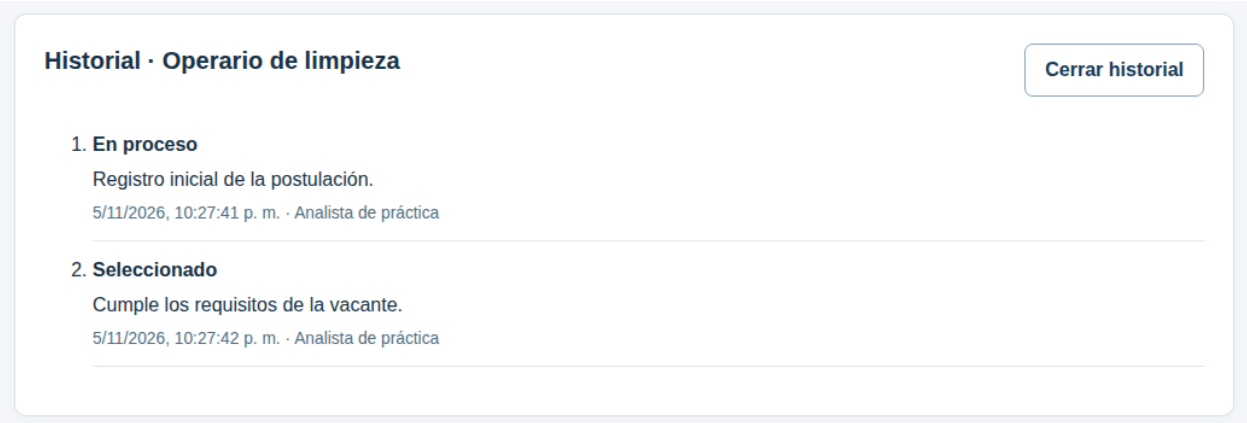
Nota. Captura real de la prueba: Lucía Prueba es una persona ficticia creada durante la ejecución. El estado Seleccionado se vuelve a consultar después de recargar la página; la prueba también modifica el teléfono y comprueba el resultado.

Consulta del historial registrado

El historial conserva la creación de la postulación y cada cambio posterior, con su observación y responsable. Esto permite reconstruir el avance de un proceso sin modificar los datos generales del candidato.

Figura 4

Detalle de los cambios registrados en la API



Nota. Ampliación de la misma captura de prueba, sin alterar los datos. Se muestran el estado inicial En proceso y el cambio a Seleccionado. Las horas corresponden al entorno de ejecución de las pruebas.

La postulación tiene su propio identificador. Si una persona se presenta a otra vacante, su nuevo proceso conserva un estado y un historial independientes. Esta separación evita confundir las decisiones tomadas para cargos distintos.

Vista móvil y permisos por rol

El diseño responde al ancho de pantalla. La prueba móvil usa una ventana de 390 × 844 píxeles y verifica navegación con el rol coordinador. Ocultar botones es una ayuda visual; la autorización real se valida nuevamente en el servidor.

Figura 5

Consulta desde una pantalla pequeña



Operación	Analista	Coordinador
Consultar y buscar	Sí	Sí
Registrar, editar y postular	Sí	No
Cambiar estado	Sí	No
Consultar historial	Sí	Sí

Nota. La API rechaza con HTTP 403 la escritura del coordinador. La ruta de registro también redirige a la consulta cuando el rol no tiene permiso.

Estándares de codificación y comentarios

Los componentes y archivos JSX usan PascalCase: FormularioCandidato, ListaCandidatos y EstadoPostulacion. Las variables y funciones usan camelCase, por ejemplo guardarCandidato e idVacante. Los hooks se invocan en el nivel superior de los componentes. ESLint revisa errores y reglas de hooks; Prettier unifica sangría, comillas y distribución del código.

Los comentarios explican decisiones que no son evidentes: el proxy de Vite mantiene el mismo origen, HashRouter permite recargar las rutas y el servicio renueva el token CSRF tras iniciar o cerrar sesión. Las clases Java siguen PascalCase; las tablas SQL usan nombres descriptivos en snake_case.

Fragmento de la conexión implementada

```
// Spring Security renueva el token al autenticar.
async login(correo, password) {
  const user = await request('/auth/login', {
    method: 'POST',
    body: { correo, password },
    form: true,
  });
  csrfToken = undefined;
  return user;
}
```

El formulario usa onSubmit y evita el envío tradicional de la página; los campos usan onChange. Los eventos reciben funciones que se ejecutan al interactuar, conforme al modelo de React (React, s. f.). Se muestran estados de carga, validación y error antes de permitir nuevas acciones.

El código completo se entrega en frontend/src y backend/src; este fragmento no reemplaza los archivos ejecutables.

Conexión con la API REST

En desarrollo, el navegador llama a /api en el puerto 5173. Vite redirige estas peticiones a Spring Boot en el puerto 8080. La respuesta JSON actualiza los componentes. El archivo .env.example documenta VITE_API_URL; el valor por defecto es /api.

Método y recurso	**Función**
GET /auth/csrf	Obtiene el token de seguridad.
POST /auth/login; GET /auth/me	Inicia y consulta la sesión.
POST /auth/logout	Cierra la sesión.
GET /candidatos?busqueda=	Consulta y filtra candidatos.
POST /candidatos	Registra una persona.
GET y PUT /candidatos/{id}	Consulta o actualiza sus datos.
GET /vacantes	Lista cargos y cupos.
GET /candidatos/{id}/postulaciones	Consulta procesos de una persona.
POST /postulaciones	Relaciona candidato y vacante.
PATCH /postulaciones/{id}/estado	Actualiza estado y observación.
GET /postulaciones/{id}/historial	Consulta cambios y responsables.

Nota. Todas las rutas tienen el prefijo /api. Los campos y permisos se detallan en docs/CONTRATO_API.md. Las mutaciones requieren token CSRF; las operaciones del dominio requieren sesión.

Las respuestas distinguen validación incorrecta (400), falta de sesión (401), falta de permiso (403), registro inexistente (404) y conflicto o duplicado (409). La interfaz comunica el problema y conserva los datos del formulario cuando guardar falla.

Datos, integridad y seguridad

Entidad	Uso en el proyecto
rol y usuario	Identidad, permisos y contraseña protegida con BCrypt.
cargo y vacante	Cargo ofrecido, cupos positivos y apertura de la oferta.
candidato	Identificación única y datos de contacto.
estado_seleccion	Catálogo: En proceso, Seleccionado y Rechazado.
postulacion	Relación única entre candidato y vacante.
historial_estado	Fecha, estado, responsable y observación de cada cambio.

El servidor valida campos obligatorios, longitudes y correo. Las restricciones de la base impiden documentos y postulaciones duplicadas. Las consultas usan parámetros; no concatenan el texto introducido por el usuario. La transacción confirma juntos el nuevo estado y su historial.

Spring Security mantiene protección CSRF para las operaciones que modifican datos (Spring, s. f.). La cookie de sesión es HttpOnly, y las contraseñas se almacenan con BCrypt. El responsable de un cambio procede de la sesión autenticada, no de un campo modificable del formulario.

Base de ejecución

El perfil de práctica utiliza H2 persistente en backend/data para facilitar la ejecución. Se incluye un perfil MySQL que requiere una base nueva y las variables DB_USER y DB_PASSWORD. Las pruebas de esta entrega se ejecutan con H2; no se presenta la configuración MySQL como una prueba realizada.

Las cuentas y datos incluidos son ficticios para clase. Antes de utilizar el sistema con personas reales se requiere configurar el despliegue, las cuentas y la protección del entorno.

Verificación de funcionamiento

La ejecución se realizó en GitHub Actions con Node.js 22, Java 17 y Chromium. Se instaló el proyecto, se compiló el front-end y se probaron las operaciones contra la API real. Las pruebas de navegador usan una base H2 aislada; no afectan una base de usuarios.

Tabla 2

Resultados de la ejecución verificada

Comprobación	Resultado y alcance
ESLint y Vite	ESLint sin errores y compilación Vite completada.
Pruebas unitarias del front-end	3 pruebas aprobadas: campos, longitudes, correo y documentos de texto.
Pruebas de integración de la API	6 pruebas aprobadas; sin fallos, errores ni pruebas omitidas. Incluye inicio de sesión y tildes UTF-8.
Pruebas de navegador	3 pruebas aprobadas en Chromium, sin fallos ni reintentos inestables.
Persistencia al recargar	La prueba vuelve a consultar la postulación y conserva el estado guardado.
Permisos y seguridad	Se comprueban 401 sin sesión, 403 por rol y 403 sin CSRF.
Validación y errores	Se comprueban campos vacíos, duplicados, ausencia de resultados y fallo de conexión.

Ejecución y registros: Abrir los resultados de verificación en GitHub

Los reportes y las capturas se incluyen en la carpeta evidencias del ZIP. Son resultados de pruebas definidas para el alcance; no equivalen a una certificación de seguridad ni a una prueba de todos los navegadores.

Manejo de fallos y experiencia de uso

Si Spring Boot está detenido o la conexión se interrumpe, la pantalla informa que no se pudo conectar y permite reintentar. El sistema no muestra un guardado exitoso antes de recibir la confirmación del servidor. Los campos tienen etiquetas asociadas y los mensajes se anuncian con roles de estado o alerta.

Figura 6

Respuesta visible ante una conexión interrumpida



Nota. En esta prueba se interrumpe deliberadamente la petición de consulta desde el navegador. La captura documenta el mensaje de error; no indica que la ejecución final haya fallado.

Para una demostración en clase, iniciar primero el backend y después Vite. Usar una cuenta de práctica según el rol que se quiera mostrar. Al terminar, cerrar sesión mediante el botón del encabezado.

Repositorio y archivos entregados

Repositorio GitHub: sena-ga7-aa4-ev03-seleccion

El repositorio contiene frontend, backend, documentación y el flujo de pruebas. Git conserva los cambios mediante commits. La compilación y las comprobaciones se pueden repetir desde la pestaña Actions. El enlace corresponde al código; no es una página pública con la API desplegada.

Versión comprobada: Ejecución #4; commit de origen 85c7dcf191e8bf2d12341aebf84025f213f27696.

Archivo o carpeta	Contenido
PDF de la evidencia	Portada, explicación, capturas, verificación, lista de chequeo y referencias.
proyecto/frontend	Código React, Vite, estilos, pruebas y dependencias fijadas.
proyecto/backend	API Spring Boot, esquema, configuración y pruebas.
proyecto/docs	Contrato REST y decisiones del proyecto.
evidencias	Capturas y reportes de la ejecución.
ENLACE_GITHUB.txt	Enlace directo al repositorio y a la comprobación.
historial.git.bundle	Copia del historial Git local de la preparación del proyecto.

El ZIP no incluye node_modules ni carpetas temporales. Las dependencias se reconstruyen con npm ci y Maven. El historial público de GitHub registra la publicación y las revisiones del código; el bundle conserva la preparación local.

Paso a paso para ejecutar y explicar

1. Descomprimir

Extraer el ZIP y abrir la carpeta proyecto en Visual Studio Code.

2. Revisar herramientas

Usar Node.js 22.12 o superior, Java 17 y Maven 3.9 o superior.

3. Iniciar la API

En una terminal entrar a backend y ejecutar mvn spring-boot:run. Esperar el inicio en el puerto 8080.

4. Iniciar React

En otra terminal entrar a frontend, ejecutar npm ci y luego npm run dev. Abrir http://localhost:5173.

5. Ingresar

Usar analista@demo.local y Practica2026! para practicar escritura; coordinador@demo.local usa la misma clave inicial y solo consulta.

6. Demostrar el proceso

Registrar una persona ficticia, abrir su detalle, escoger una vacante, crear la postulación y cambiar su estado con observación.

7. Consultar y comprobar

Abrir Ver historial, recargar, buscar la persona y editar su teléfono. Recorrer Inicio, Vacantes y Mi perfil.

8. Revisar permisos

Cerrar sesión e ingresar como coordinador. Comprobar que puede consultar y no tiene acciones de escritura.

9. Repetir verificaciones

Frontend: npm run lint, npm test y npm run build. Backend: mvn test. README explica las pruebas de navegador con base aislada.

Las cuentas se crean únicamente al inicializar una base vacía. Reiniciar conserva los datos de H2; borrar backend/data elimina los registros de práctica. El README también explica el perfil opcional MySQL.

Lista de chequeo de la evidencia

Tabla 3

Autoevaluación frente al instrumento GA7-AA4-EV03

Indicador	Sí	No	Observación verificable
1. El diseño front-end cumple con los prototipos realizados.	X		Tabla 1, figuras 2-5 y componentes. Se documentan las adaptaciones del prototipo genérico al alcance EV02.
2. Entrega todos los archivos del proyecto y enlace de repositorio.	X		ZIP con frontend, backend, configuración, pruebas y documentación; enlace en esta evidencia y ENLACE_GITHUB.txt.
3. La navegación del front-end funciona correctamente.	X		Pruebas de navegador y reportes: acceso, rutas, registro, edición, consulta, postulación, historial y salida.
4. Cumple con los requisitos especificados inicialmente.	X		Registro, búsqueda, consulta, estados por postulación, roles, historial y mensajes. Tabla 1 y pruebas.
Solicitud adicional: React, Vite, comentarios, PascalCase, Git y Spring Boot.	X		Archivos fuente, package.json, clases de API, comentarios, configuración ESLint/Prettier e historial de commits.

Nota. Autoevaluación de la entrega; no reemplaza la valoración ni la firma de la instructora. El alcance es web adaptable. La verificación usa H2 y la API real; el perfil MySQL está disponible como configuración adicional.

Conclusiones

El diseño EV02 se convierte en una aplicación conectada: una acción del usuario produce una petición al servidor, el servidor aplica reglas y la interfaz presenta el resultado. La separación en componentes, las pruebas y el historial de versiones permiten revisar y continuar el proyecto de forma organizada.

Referencias

Ciro Botero, L. T. (2026a). *Diseño de la interfaz web: GA5-220501095-AA1-EV04* [Documento del proyecto formativo]. Servicio Nacional de Aprendizaje.

Ciro Botero, L. T. (2026b). *Verificación de procedimientos para la definición de componentes front-end de la aplicación: GA7-220501096-AA4-EV02* [Documento del proyecto formativo]. Repositorio EV02

React. (s. f.). *Responding to events*. https://react.dev/learn/responding-to-events

Servicio Nacional de Aprendizaje. (s. f.). *Guía de aprendizaje 7 e instrumento de evaluación: GA7-220501096-AA4-EV03* [Material de formación y lista de chequeo proporcionados para la actividad].

Spring. (s. f.). *Cross Site Request Forgery (CSRF)*. Spring Security. Documentación de protección CSRF

Vite. (s. f.). *Getting started*. https://vite.dev/guide/

Presentación académica: tamaño carta, márgenes de 2,54 cm, Times New Roman de 12 puntos, interlineado doble en el cuerpo, numeración de páginas, tablas y figuras tituladas y referencias con sangría francesa. Las tablas y los fragmentos de código usan tamaños ajustados para mantener su legibilidad.